

Methodology

1 Research Framework

This research develops one coherent system for defending persistent LLM-agent memory against poisoning, rather than a set of separate projects. The system has four interlocking components.

A **unified, attack-agnostic benchmark** (Section 5) represents heterogeneous memory-poisoning attacks under a single memory representation, agent-memory environment, threat model, and evaluation protocol, so attacks as different in mechanism as AgentPoison, MINJA, and sleeper-style dormant poisoning can be compared on equal footing.

A **common, lifecycle-aware defense** (Section 6) addresses three research gaps — provenance and governance, self-reinforcing propagation, and sleeper/dormant poisoning — as a single integrated architecture.

Attack-origin attribution and forensic reconstruction (Section 7) extends the defense beyond binary poisoned/clean classification toward reconstructing where an attack entered the memory lifecycle, how it propagated, where it activated, and what it affected, with explicit, calibrated uncertainty where evidence is incomplete.

Adaptive, risk-based hardening (Section 8) closes the loop by using attribution output to increase scrutiny selectively around implicated pathways, rather than applying uniform restriction across the whole memory subsystem.

These components are bound together by two proposed learned components, detailed in Sections 9–11: a Graph Neural Network (GNN) for relational structure, and a Gated Linear Network (GLN) for continual risk adaptation. Their proposed combination — unified benchmark, common lifecycle-aware defense, attack-origin attribution, and adaptive hardening, instantiated through a GNN + GLN hybrid — is the organizing hypothesis of the project and is treated throughout as an approach to be experimentally validated, not an established result. Specific architecture variants named later (GCN, GraphSAGE, GAT for the GNN; specific GLN configurations) are candidates whose selection is an empirical outcome of Section 12.

The document proceeds in build order: data preparation and representation (Sections 2–4); the unified benchmark and common defense (Sections 5–8); the learned components and their proposed combination (Sections 9–11); and the development sequence, experimental protocol, evaluation, ablations, and generalization tests (Sections 12–16) that determine, empirically, how much of the proposed architecture is retained.

2 Data Preparation and Normalization

2.1 Purpose and Scope

Every downstream component — the unified memory representation, the lifecycle graph, the attack benchmark’s seeding procedure, and the GNN/GLN components —

depends on a single, normalized, provenance-annotated substrate of benign memory content. This section describes how that substrate is produced from four heterogeneous source datasets: LoCoMo, LongMemEval, MSC (Multi-Session Chat), and Conversation Chronicles. This stage does not generate attack content and does not run the benchmark; its output is a clean, normalized, traceable memory-record corpus consumed by later stages.

Four properties govern this stage and are carried forward as constraints on every later component that touches memory data: raw source data is treated as immutable and is never edited in place; every normalized record retains a traceable link back to its exact source (dataset, split, conversation/session identifier, turn or QA index); no field is assumed to exist in every dataset, and presence is verified per dataset before it is relied upon; and every transformation is deterministic and reproducible under the project’s fixed master seed and configuration version, so re-running this stage against the same source snapshots yields identical output. This traceability requirement is a direct prerequisite for attack-origin attribution (Section 7), which depends on tracing a flagged memory through its full derivation history, including the preparation stage itself.

2.2 Per-Dataset Inspection

Before cleaning or normalization, each dataset is inspected independently, rather than assuming a common schema across sources. Inspection establishes, per dataset: the unit that constitutes a candidate memory entry (a conversational turn, a QA pair, a session summary); what session- and conversation-level identifiers exist and how they nest; what timestamp or ordering information is present, and whether it is a genuine recorded time or merely relative ordering; whether speaker/role labels exist and how many distinct roles appear; whether any existing metadata could plausibly map to provenance information; and what encoding, language, or formatting irregularities appear in a representative sample.

Based on prior survey work, the four datasets are expected to differ along these dimensions as follows:

- **LoCoMo** supplies long, multi-session synthetic conversations (10-50 conversations, roughly 200 QA pairs per conversation) with session-level structure and an associated QA/evaluation layer. LoCoMo’s answer/evaluation layer has documented data-quality concerns; these are treated as quality issues to detect and flag during inspection rather than assumptions to inherit uncritically, and are not quantified numerically here in the absence of a directly citable source within this document.
- **LongMemEval** provides long-horizon, multi-session evaluation data designed to stress long-term memory recall; its cleaned distribution is inspected for consistent turn-level structure before use.
- **MSC (Multi-Session Chat)**, accessed via ParlAI, provides real human-human dialogue rather than LLM-generated conversation, making it a useful source of naturally occurring conversational data. This distinguishes MSC structurally from the other three sources; it is not treated as inherently more trustworthy or more accurate than synthetic sources, only as differently sourced, and its

provenance fields are populated accordingly rather than assigned an elevated trust prior by default. Its inspection focuses on how ParlAI’s session-episode structure maps onto this project’s session identifier field.

- **Conversation Chronicles** contributes long-term, relationship-evolving multi-session dialogue with inter-session time-interval annotations, inspected for whether those intervals can inform realistic relative timestamp spacing during later aged-memory seeding.

Beyond these four core memory datasets, the project draws on a broader resource inventory. Each resource’s source, version, licensing, access status, and preparation status are tracked in a resource registry. The inventory below states, per resource, its preparation action and its role in the benchmark.

Core Memory Datasets (clean and normalize into the unified memory substrate of Section 3): LoCoMo, LongMemEval, MSC, and Conversation Chronicles, detailed above. For LongMemEval specifically, the project’s process record notes the `m_cleaned` distribution as not yet downloaded at this stage and identifies two source-corrupted records as an outstanding issue; for MSC, the process record notes the dataset’s licensing status as unclear rather than resolved; for Conversation Chronicles, a sampling issue is likewise recorded as outstanding. None of these three issues is presented here as resolved.

Task / Workload Resources (inspected and normalized for their purpose-specific role rather than folded into the memory substrate): API-Bank (inspect and normalize for tool/API task representation), ToolBench (inspect and normalize for tool-use workloads), StrategyQA (inspect and normalize for reasoning/task evaluation), WebShop (inspect and normalize for web/task interaction workloads), SWE-bench Verified (inspect and normalize for software-engineering agent workloads), τ -bench / τ^2 -bench (inspect and normalize for interactive agent/tool-use evaluation), and EHRAgent (inspect and normalize for domain-specific agent/task evaluation). MIMIC-III and eICU, which underlie EHRAgent’s clinical setting, are inspected and prepared only if confirmed necessary and usable, and are not represented as definitively included experimental datasets given their credentialed-access requirement. Of this group, API-Bank, StrategyQA, SWE-bench Verified, and τ -bench/ τ^2 -bench reached a prepared status at this stage; ToolBench, WebShop, and EHRAgent were inspected, with their accessibility and reproducibility documented rather than preparation completed.

Sleeper / Dormant-Poison Resources (prepare generation and evaluation resources rather than benchmark-ready data): Hidden in Memory was obtained and inspected as a sleeper-evaluation resource. The Sleeper Dataset Generator’s available sleeper-agent few-shot templates were retrieved, and a reproducible generator configuration/template system — supporting controlled generation of sleeper scenarios with specified triggers, contexts, labels, and reproducible seeds — was prepared; the final experimental sleeper dataset itself had not yet been generated at this stage, and is not presented as available.

Attack / Benchmark Resources (prepare attack specification or scenario data rather than a runnable implementation, per the reconstruction approach of Section 5.1): MPBench (benchmark/attack scenarios), AgentPoison, MINJA, MemoryGraft, and FARMA (attack specification/data). DSRM’s attack specification/data was like-

wise prepared, but its implementation and resource availability remain unverified, consistent with Section 5.1’s treatment of DSRM as lacking a released downloadable artifact.

Security / Comparison / Evaluation Resources (prepare comparison and evaluation resources rather than benchmark-substrate data): MemSecBench, MEMSAD, and MemAudit (comparison/evaluation resources), A-MemGuard (baseline/defense comparison resources), ASB (evaluation resources), AgentDojo (evaluation environment/resources), and InjecAgent (injection/evaluation resources).

Across all categories, the project does not blindly discard malformed or unusual records. Each record is inspected and classified — as valid, repairable, flagged, or genuinely unusable — before any removal, and removed or transformed records are preserved and documented with their reason and provenance rather than silently deleted; this classification is part of what distinguishes ordinary data noise from the malicious patterns the benchmark is designed to detect, and is reflected in the cleaning categories of Section 2.3.

Each inspection produces a per-dataset schema report — field inventory, observed value ranges, missing-field rate, structural quirks — that is the direct input to the mapping in Section 2.4.

The project does not apply one universal cleaning or normalization rule to every resource. Each resource is inspected and prepared according to its intended role in the benchmark. The four core memory datasets are normalized into the unified memory substrate described in Section 3, while task, attack, sleeper, and comparison resources retain representations appropriate to their respective experimental roles. Source provenance, accessibility, preparation status, and any known limitations are preserved so that unavailable, unverified, or conditionally usable resources are not silently treated as experimentally available.

2.3 Cleaning and Validity Filtering

Cleaning is applied per dataset, using rules justified by that dataset’s own inspection report; no rule is applied uniformly across all four sources without confirming its appropriateness. The categories applied, where relevant, are:

1. **Structural validity.** Records missing a required minimal field (raw text content and a resolvable session/conversation identifier) are excluded rather than repaired, since fabricating an identifier would break traceability.
2. **Encoding and formatting repair.** Non-UTF-8 sequences, broken markup, and control characters are normalized or stripped — a mechanical step distinguished from content-level filtering.
3. **Duplicate detection.** Exact duplicates within a conversation are removed. Near-duplicate detection uses embedding similarity under the same semantic encoder committed for the benchmark’s retrieval index, so deduplication and later retrieval share one notion of “same content.” Discarded near-duplicates are logged by source identifier.
4. **Known data-quality flagging.** Where a source has a documented quality issue, affected records are tagged with a `data_quality_flag` rather than silently

corrected, so any later result can be checked for dependence on flagged content.

5. **Length and degeneracy filtering.** Empty, truncated, or template-artifact entries surviving a dataset’s own generation process are excluded.

Every exclusion is recorded in a per-dataset removal log (source id, removal reason, rule applied), so the eventual seed corpus’s provenance is auditable back through cleaning, not only back to the raw file — a property Section 7 relies on when determining whether a flagged entry’s anomalous characteristics originated in preparation rather than in an attack.

2.4 Normalization into the Unified Memory Record

Cleaned records from all four datasets are mapped into one common schema, introduced fully in Section 3. Fields are populated by one of four explicitly distinguished mechanisms — **source-provided**, **benchmark-generated** (deterministic simulation under the fixed seed), **inferred** (derived from source structure via a documented rule), or **model-predicted** (produced at runtime by a defense component, not during preparation) — and the mechanism used is recorded per field per record.

- **memory_id**, **content**, **source_dataset**, **source_ref** are source-provided or mechanically derived from source identifiers.
- **session_id** is mapped from the dataset’s own session/episode structure where present (LoCoMo, MSC, Conversation Chronicles); where absent, it is assigned sequentially and marked benchmark-generated.
- **timestamp** distinguishes three cases (detailed in Section 3.2): genuine source timestamps, source-relative ordering, and benchmark-assigned timestamps. A record’s timestamp is never presented as genuine when it was in fact benchmark-assigned.
- **source_role** is mapped from source speaker/role labels.
- **trust_score** is drawn from a fixed prior distribution assigned by source category, not by dataset as a whole, and is explicitly a prior assumption rather than a ground-truth judgment (Section 3.3).
- **derivation_parents** is populated only where a dataset’s own structure supports it; otherwise left empty rather than fabricated.
- **data_quality_flag** is set during cleaning (Section 2.3).
- **embedding** is computed under the fixed semantic encoder used throughout the benchmark.

This mapping is applied per dataset, since the population strategy for several fields depends on structure each dataset’s inspection already established; where a dataset does not support a field’s intended derivation, the harness’s documented default is used and logged.

2.5 Cross-Dataset Consistency Checks and Output Artifacts

After normalization, a validation pass checks the combined corpus for internal consistency: schema conformance across all required fields and types; embedding va-

lidity (no zero-norm or NaN vectors); session-identifier uniqueness within, though not necessarily across, source datasets; and a distributional check confirming the combined corpus’s trusted/untrusted composition reflects actual source composition rather than an arithmetic split applied without regard to it. Records failing validation are excluded and logged.

This stage produces four artifacts, each traceable to its inputs: per-dataset schema reports (2.2); per-dataset cleaning and removal logs (2.3); the unified, normalized memory-record corpus in the schema of Section 3; and a validation report. These artifacts — not the raw source files, which remain untouched and retained separately for audit — are what the unified memory representation (Section 3), lifecycle graph (Section 4), and attack benchmark (Section 5) consume.

3 Unified Memory Representation

3.1 Why a Common Representation Is Required

The four source datasets differ in structure, timing information, role semantics, and metadata completeness. A single record schema is constructed as the interface between data preparation and every subsequent component, extending a core provenance record — {source, session_id, source_role, timestamp, trust_score, derivation_parents} — with the additional fields a benign or attacker-influenced entry may carry across its lifecycle.

Field	Description	Origin
memory_id	Unique record identifier	Source-derived
content	Text content	Source-provided, cleaned
source_dataset	Originating dataset	Source-provided
source_ref	Original conversation/session/turn identifier	Source-provided
session_id	Project-level session grouping	Source-mapped or benchmark-generated
timestamp / event_order	Position in real or simulated timeline	Source-provided, source-relative, or benchmark-generated (Section 3.2)
source_role	Producing role: user, agent, trusted_document, web, tool_output	Source-mapped
trust_score	Prior security assumption, not ground truth	Assigned by source category (Section 3.3)
derivation_parents	Prior entries this one derives from	Source-supported or empty
retrieval_history	Log of retrieval events referencing this entry	Benchmark-generated at runtime

Field	Description	Origin
propagation_history	Log of downstream entries derived from this one	Benchmark-generated at runtime
security_state	Current classification: normal, monitored, hardened, quarantined	Model-predicted, updated at runtime
data_quality_flag	Marks entries with a documented source-quality issue	Assigned during cleaning
embedding	Fixed-dimensional semantic vector	Benchmark-computed
attack_label / poison_status	Ground-truth label, attack-injected entries only	Benchmark-generated, attack phase only

No dataset is assumed to supply every field. Fields absent from a given source are populated using the mechanisms of Section 2.4, and that mechanism is retained as metadata on the field itself rather than merged indistinguishably into the value.

3.2 Timestamps: Source, Relative, and Simulated

Because propagation depth over time, dwell time before sleeper detection, and cross-session transfer all depend on temporal ordering, timestamp provenance is a first-class distinction. Three categories are maintained separately for every record: **genuine source timestamps**, used only where a dataset provides an actual recorded time; **source-relative ordering**, where a dataset provides sequencing or inter-event intervals without an absolute date (e.g., Conversation Chronicles), which anchors relative spacing but not calendar time; and **benchmark-assigned timestamps**, produced deterministically by the harness’s timeline-distribution procedure when no usable source timing exists. Every timestamp field is accompanied by which category produced it, and analyses depending on absolute time restrict themselves to records where that assumption is valid, or flag explicitly where it is not.

3.3 Trust Scores Are Priors, Not Ground Truth

`trust_score` represents a prior security assumption assigned at admission time — reflecting, for instance, that a `trusted_document`-sourced entry is a priori more likely to be reliable than a `web`-sourced one — and is not a claim about whether a given memory’s content is actually true or actually malicious. This distinction matters throughout: provenance-based trust weighting (Section 6.1) uses `trust_score` to modulate influence, but attribution (Section 7) and sleeper detection (Section 6.3) must still be able to identify high-trust-prior entries that are nonetheless compromised, and low-trust-prior entries that are nonetheless benign.

4 Memory Lifecycle and Graph Construction

4.1 Lifecycle

Every memory record’s life is modeled through: creation → admission/write → storage → retrieval → reuse → modification/derivation → propagation → activation → downstream influence → quarantine/deletion/recovery. Each stage is a point at which the common defense (Section 6) can observe, evaluate, and act, and each stage produces an event logged in the benchmark’s append-only event log — the raw material from which the memory graph is built.

4.2 Graph Construction

The evolving memory environment is represented as a graph wherever available data supports the relationship in question; edges are never invented to fill out a desired structure. **Memory records are the primary nodes.** Interactions, sessions, queries, episodes, and agent actions may appear as auxiliary nodes where their relationships to memory records carry security-relevant information (for example, a session node linking several memories written within it, supporting cross-entry temporal and co-occurrence analysis) — auxiliary nodes are included only where they serve this analytical purpose, not as a requirement that every possible entity type be represented.

Candidate edge types are: created-from, derived-from, retrieved-by, followed-by, reinforces, contradicts, similar-to, and temporally-related. Which edges are populated for a given record depends directly on what data preparation (Section 2) and lifecycle logging (4.1) actually captured: a derived-from edge exists only where `derivation_parents` was populated with source support or generated by an observed benchmark-runtime write-back; a similar-to edge is computed from the fixed embedding space used throughout, at a defined similarity threshold, so it is reproducible rather than incidental.

This graph is the structural substrate for propagation analysis, suspicious-memory detection, and attack-origin attribution (Sections 6.2, 6.3, 7), and the direct input to the GNN component (Section 9).

5 Unified Attack Benchmark

5.1 Common Interface, Heterogeneous Attacks

The benchmark evaluates attacks differing substantially in mechanism — Agent-Poison’s backdoor-style memory/knowledge-base poisoning, MINJA’s query-only progressive injection, MemoryGraft’s and FARMA’s respective mechanisms, sleeper/dormant poisoning’s delayed-trigger design, and MPBench-style templated scenarios — under one common experimental interface. Each attack is implemented as an attack module conforming to a standardized attack specification containing: an attack identifier; attacker capability and access level (e.g., query-only versus privileged write access); injection channel; target; payload; trigger condition, if any; attack duration (single-shot versus multi-turn versus persistent); and

intended malicious effect. Each module exposes a `run_attack()` entry point operating against the common memory representation (Section 3) and lifecycle (Section 4.1), preserving the attack’s original threat model rather than flattening it to a lowest common denominator. Where an attack’s original implementation does not provide a directly reusable `run_attack()` — as is the case for DSRM and MPBench, which exist as taxonomy/methodology without a released downloadable artifact — it is reconstructed from the attack’s published methodology as a declarative payload generator conforming to the same specification, and this reconstruction is explicitly disclosed rather than presented as equivalent in evidentiary weight to an attack with an original public implementation. No implementation is claimed to exist where it does not.

5.2 Evaluation Conditions

Every attack module is evaluated under a shared protocol: a clean baseline establishing utility and false-positive floors; an attack-without-defense condition establishing baseline vulnerability; and attack-with-defense conditions (Section 13) across the common defense’s components, standalone and combined. All conditions share the same memory substrate (Sections 2–3), lifecycle instrumentation (Section 4.1), and metric family (Section 14), so attack effectiveness and defense effectiveness are measured on a common, comparable footing.

6 Common Lifecycle-Aware Defense

The defense is one integrated architecture addressing three research gaps together. All three pillars operate over the unified memory representation (Section 3) and lifecycle graph (Section 4.2).

6.1 Provenance and Governance

The system maintains, for every memory entry, its origin, producing session, creation time (Section 3.2), source category, derivation parents, modification history, retrieval/use history, and current trust/security state. This governs admission, ongoing trust weighting, verification requirements, and lifecycle actions (quarantine, deletion, recovery). Provenance is treated throughout as security evidence, not proof of truth: a memory originating from a legitimate, high-trust-prior interaction may still contain malicious or incorrect content.

6.2 Self-Reinforcing Propagation Monitoring

This pillar models the possibility that poisoned content spreads through the agent’s own re-ingestion of its outputs: an initial poisoned entry is retrieved, influences a new generated entry, that entry is written back and reinforces the original, and the corrupted lineage expands. Monitored signals include propagation depth and breadth over the derivation graph, derived-memory count from a given ancestor, retrieval frequency, cross-session propagation, reinforcement patterns, contradiction patterns,

and downstream behavioral influence. Suspicious propagation triggers the containment actions defined in Section 8, rather than only flagging the individual entry.

6.3 Sleeper / Dormant Poison Detection

Some poisoned entries appear benign under ordinary inspection and activate only under a specific future query, context, role, or event sequence; static, single-context classification cannot reliably surface these, and text inspection alone is not assumed sufficient. Candidate memories are instead evaluated across three context classes: **normal contexts**, in which the memory is expected to remain benign across repeated ordinary use; **unrelated contexts**, plausible but non-triggering contexts that should likewise leave the memory’s behavioral influence unchanged; and **trigger contexts**, contexts specifically associated with the suspected sleeper attack. A memory is flagged as a sleeper candidate when its behavioral influence is stable across normal and unrelated contexts but shifts sharply under a trigger context — the detection target is this context-dependent divergence, distinguished explicitly from static maliciousness detectable from content alone.

7 Attack-Origin Attribution and Forensic Reconstruction

Two concepts are distinguished throughout this section and are not conflated. The **attack source** is the input, entity, or delivery mechanism responsible for generating or delivering malicious content — a webpage, a document, a tool output, a crafted user turn. The **attack origin** is the first point within the agent’s memory lifecycle at which that malicious information becomes persistent or meaningfully enters the memory system — the specific write event, memory, or session. This methodology’s attribution objective is primarily the identification of attack origin within the memory lifecycle and the reconstruction of its downstream lineage; it does not attempt, and does not claim the ability, to establish the real-world identity of an attacker.

Where a suspicious memory or propagation pattern is flagged (Sections 6.2–6.3), reconstruction proceeds: (1) identify the suspicious node in the memory graph; (2) inspect its provenance record; (3) inspect neighboring graph nodes; (4) trace parent/derived relationships along the derivation graph; (5) analyze temporal ordering of the candidate lineage, respecting the timestamp-provenance distinctions of Section 3.2; (6) generate candidate origin nodes; (7) assign each candidate an attribution confidence score; (8) reconstruct the propagation path from candidate origin(s) to the flagged node; (9) identify the likely activation point, where relevant; and (10) record downstream impact.

Six outputs are kept explicitly distinct: **origin** (where compromise most plausibly began), **entry mechanism** (how it entered — which write channel or attack modality), **propagation path**, **activation point**, **downstream impact**, and **attribution confidence** (a calibrated score reflecting evidence completeness, not a certainty claim). Where evidence is incomplete, the output is a ranked list of candidate origins with confidence scores rather than a forced single answer.

This process depends on the traceability guarantees established during data prepa-

ration (Section 2.1) and the provenance schema (Section 3): attribution is only as reliable as the lineage information available, which is why immutable raw-data preservation and per-transformation traceability were established as constraints from the earliest stage rather than added later.

8 Adaptive, Risk-Based Hardening

Attribution output (Section 7) feeds a closed-loop adaptive defense rather than terminating in a report. Pathways are assigned a risk tier, and protection scales with that tier: **low risk** receives normal validation; **medium risk** receives stronger provenance verification and increased monitoring; **high risk** receives stricter validation, restricted memory writes, and quarantine of the specific suspicious memories involved; **critical risk** results in temporary isolation of the affected pathway, quarantine of affected memories, and triggers deeper forensic analysis (Section 7).

De-escalation is evidence-based, not time-based: a pathway returns toward normal validation only when continued clean observation and the absence of corroborating evidence support reduced risk, not simply because a fixed interval has elapsed. Escalation, similarly, is triggered by new corroborating evidence rather than a single ambiguous signal. The closed loop is: detect → attribute → identify vulnerable pathway → harden → monitor → reassess risk → adapt, with reassessment explicitly conditioned on accumulated evidence at each cycle.

9 GNN-Based Relational Modeling

9.1 Motivation

Several signals the defense pillars depend on — propagation depth and breadth, derivation lineage, cross-entry reinforcement and contradiction, candidate-origin identification — are relational and cannot be recovered from a memory record in isolation, only from its position in the memory/interaction graph (Section 4.2). This motivates a Graph Neural Network as the primary candidate for the graph-based component.

9.2 Inputs and Task

The GNN operates over the graph defined in Section 4.2. Node features draw from the unified memory representation: semantic embeddings, provenance fields, temporal features (respecting Section 3.2’s source/relative/benchmark-generated distinction), retrieval frequency, propagation counts, `trust_score`, source category, and contradiction signals where computed. Edge features include relationship type, temporal distance, and interaction/retrieval frequency between connected nodes.

Through message passing, the GNN produces contextual, relationship-aware node representations, from which several outputs are investigated: suspicious-memory detection, propagation detection, memory-influence estimation, and graph-level risk estimation for a pathway (feeding Section 8’s risk-tier assignment). With respect to

attribution specifically, the GNN’s role is deliberately scoped: it identifies and ranks suspicious nodes and relationships and generates **candidate origin nodes with relational evidence**; it does not, alone, determine a final origin. Final attribution (Section 7) combines this relational evidence with provenance, temporal, and lifecycle evidence — the appropriate description of the GNN’s contribution is attack-origin candidate ranking, not origin identification.

9.3 Candidate Architectures

GCN, GraphSAGE, and GAT are treated as candidate architectures. Final selection is an experimental outcome of Section 12, based on measured performance, scalability to the benchmark’s graph sizes, and interpretability of resulting risk signals — attribution in particular benefits from architectures whose aggregation or attention weights admit inspection. No architecture is presumed superior in advance.

10 GLN-Based Continual Security Modeling

10.1 Motivation

The memory environment is not static: writes, retrievals, and propagation events occur continuously, and a defense requiring full retraining after every event is impractical. This motivates a Gated Linear Network as the candidate continual/online-learning component, given GLNs’ characteristic support for local, incremental updates.

10.2 Inputs and Task

The GLN operates on a sequence of memory-security events (writes, retrievals, propagation steps, flags raised by Sections 6–7) and associated contextual security features, which may include GNN-derived representations where the hybrid configuration (Section 11) is in use. For each incoming event, the GLN produces a current risk estimate and a proposed security action, then incorporates the event through a local/online update, producing an updated risk model for the next event. Investigated outputs include a risk score, a trust update, a security-state classification, a hardening-level recommendation (feeding Section 8), a monitoring-intensity recommendation, and a quarantine recommendation.

10.3 Scope of Claims

The GLN is investigated for its potential to support continual adaptation in a changing memory environment; no claim is made in advance that it will outperform conventional online-learning baselines or a GNN-only configuration. That comparison is an explicit part of the evaluation design (Sections 12, 14, 15).

11 Proposed GNN + GLN Hybrid

The central proposed architecture combines the components of Sections 9-10: the memory graph is processed by the GNN to produce relational security representations — suspicious-memory, propagation, and origin-candidate signals — aggregated into a security-state representation; this state, together with the sequential stream of new memory-security events, is passed to the GLN, which continually updates risk estimates and proposes adaptive security decisions; those decisions determine the risk-tier actions of Section 8, and resulting new memory events feed back into the graph, closing the loop: observe → represent → detect → attribute → decide → harden → observe again.

The hybrid is investigated under three configurations, compared against each other and against the single-component baselines of Sections 9 and 10:

Configuration A — GNN-only. Memory graph → GNN → security outputs (suspicious-node scores, propagation signals, origin candidates), used directly without a continual-adaptation stage.

Configuration B — GLN-only. Sequential security events → GLN → adaptive security outputs, without graph-derived relational features.

Configuration C — Sequential GNN → GLN hybrid. Memory graph → trained GNN → relational security representation → GLN → continual risk estimate and adaptive decision. This is the primary proposed configuration.

Where computationally and methodologically justified, joint or combined updating of the GNN and GLN components may additionally be investigated; joint training is not assumed necessary or superior to the sequential configuration and is treated as a further experimental question rather than a default. The purpose of comparing these configurations is to determine empirically whether the hybrid provides benefit beyond either component alone — this is presented as a research hypothesis to be tested (Sections 12, 15), not as an established result, and the possibility that a simpler configuration matches or outperforms the full hybrid on some or all metrics is a legitimate experimental outcome.

The unified memory representation and provenance schema (Sections 2-3) were constructed specifically to support this architecture: the field-mechanism separation, the retained derivation graph, and the retrieval/propagation history fields are what graph construction (4.2), the GNN’s features (9.2), and the GLN’s event stream (10.2) require as input.

12 Model Development and Training Methodology

Development proceeds in a fixed sequence so each component’s contribution can be isolated before combination: (1) prepare clean, normalized datasets (Section 2); (2) construct the unified memory representation (Section 3); (3) construct the lifecycle graph and temporal features (Section 4); (4) establish conventional, lightweight baselines (logistic regression, random forest, gradient boosting, MLP) on the unified representation’s non-graph features, providing a concrete reference point for the value

added by graph and continual-learning components; (5) develop and evaluate the GNN model (Configuration A); (6) develop and evaluate the GLN model (Configuration B); (7) develop and evaluate the sequential GNN $\rightarrow$ GLN hybrid (Configuration C), and, where justified, a joint-updating variant; (8) integrate the selected model(s) into the common defense (Section 6), connecting outputs to provenance/governance, propagation monitoring, sleeper detection, attribution, and adaptive hardening; (9) integrate the attack modules (Section 5); (10) run the full evaluation protocol (Section 13); (11) run ablations (Section 15); (12) run cross-attack generalization tests (Section 16).

Split methodology. Training/validation/test separation is enforced at the level of connected components in the lineage graph, not individual memory records. Where memory records are connected through derivation, parent/child relationships, propagation, or direct graph connectivity, the entire connected component (or an appropriately bounded lineage subgraph, where full-graph connectivity would make splitting impractical) is assigned to a single split; a record and its derivation ancestors or descendants are never permitted to fall on opposite sides of a split, since this would leak lineage information the GNN or attribution mechanism could otherwise exploit only at evaluation time. For the GLN and continual-learning evaluation specifically, splits are chronological rather than random: past events form the training stream, later events form the validation stream, and future events form the test stream, reflecting the requirement that continual adaptation be evaluated as a genuine online-prediction problem rather than an i.i.d. classification task.

Attack-free baseline training is used where a component’s intended function is to detect deviation from normal behavior rather than recognize attack-specific signatures; both attack-agnostic training (exposure to a diverse attack mixture) and attack-specific ablations are evaluated where feasible, since the generalization question (Section 16) requires distinguishing what the system has learned from what it has memorized. All training is conducted under the project’s fixed seed and configuration versioning, with model checkpoints and configuration versions tracked for reproducibility. This methodology does not specify hyperparameters or anticipated results in advance.

13 Experimental Protocol

Each (attack, defense-configuration) pair is evaluated under: (A) a clean environment, establishing baseline utility with no attack present; (B) attack without defense, establishing baseline vulnerability; (C) attack with individual defense components in isolation (provenance-only, propagation-monitoring-only, sleeper-detection-only, attribution-only, adaptive-hardening-only); (D) the full common defense, combining all pillars with the selected model configuration from Section 12; (E) adaptive hardening evaluated specifically for its effect on repeated-attack attempts through a previously implicated pathway; (F) cross-attack conditions, evaluating a given configuration against attack types different from those emphasized during development; and (G) unseen-attack evaluation where feasible, using an attack module withheld entirely from development.

14 Evaluation

Evaluation spans five metric families, reported together rather than in isolation, since a defense degrading utility unacceptably is not considered successful regardless of its security metrics.

Security: attack success rate, poisoning-detection rate, false-positive rate, persistence, propagation depth and breadth, containment rate, sleeper-activation detection rate, time-to-detection, time-to-containment.

Attribution: origin-identification accuracy, entry-point localization accuracy, propagation-path reconstruction accuracy, temporal localization accuracy, and attribution confidence calibration.

Adaptive defense: reduction in repeated attacks through a previously hardened pathway, reduction in propagation following hardening, time-to-harden, recovery/de-escalation behavior when risk subsides, false-blocking rate.

Model: accuracy, precision, recall, F1, AUROC/AUPRC where applicable, online-adaptation performance (GLN specifically), computational cost.

Utility: clean-task performance, memory retrieval quality, response quality, latency, storage overhead, inference overhead, update cost.

The security-utility tradeoff is reported as a primary result. A configuration achieving strong security metrics by substantially degrading utility — for instance, by blocking most legitimate memory operations — is not considered successful under this methodology.

15 Ablation Studies

Systematic ablations remove one component at a time from the full system: no provenance/governance, no propagation monitoring, no sleeper detection, no attribution, no adaptive hardening, no GNN, no GLN, GNN-only, GLN-only, and the full sequential hybrid. These are designed to determine empirically which component contributes most to overall effectiveness; whether the GNN measurably improves relational/propagation detection over non-graph baselines; whether the GLN measurably improves continual adaptation over static retraining; whether the hybrid provides benefit beyond either component alone; whether adaptive hardening measurably reduces repeated attacks relative to static defense; and whether attribution measurably improves downstream defense effectiveness or is primarily of forensic value.

16 Cross-Attack Generalization

A central research question is whether the proposed defense learns general security principles — properties of provenance, propagation, and context-dependent activation that hold across attack mechanisms — rather than attack-specific signatures effective only against attacks emphasized during development. This is evaluated

by comparing performance on attacks present during development against attacks held out entirely, across different mechanisms (backdoor-style, query-only injection, delayed-trigger sleeper attacks), different entry points and write channels, and, where feasible, different memory environments or task domains. The question posed is whether the combination of explicit provenance, graph-based relational modeling, continual risk adaptation, propagation monitoring, contextual sleeper detection, attack-origin attribution, and adaptive hardening — this methodology’s core hypothesis (Section 1) — provides more generalizable protection than defenses operating only at the memory-write or retrieval stage. This document does not assert that hypothesis as confirmed; it defines the experimental design (Sections 12–16) by which it is to be tested.